

Doubly Linked List – Insert At Beginning –Space Complexity and Algorithm.

Program :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    if (head == nullptr)
    {
        newNode->prev = nullptr;
        newNode->next = nullptr;
        head = newNode;
        cout << data << " inserted at the beginning." <<
endl;
    }

    else
    {
        newNode->prev = nullptr;
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
        cout << data << " inserted at the beginning." <<
endl;
    }
}
```

```
..... •
case 1:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtBeginning(data);
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary space used by an algorithm, excluding the space taken up by the inputs.

For the `insertAtBeginning` function, the only new memory allocation is for `newNode`:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

The size of `newNode` is `sizeof(Node)`, which is a **constant** value. It does not depend on the number of nodes already in the list. Therefore, the auxiliary space complexity is:

- **Auxiliary Space:** $O(1)$
-

Input Space

Input space is the amount of memory used to store the input values. The definition of "input" can vary, leading to different analyses.

1. Parameter-only Definition

This common definition considers only the space taken by the function's parameters. The function signature is `void insertAtBeginning(int data)`.

- The input is a single integer, `data`.
- An `int` occupies a fixed, constant amount of memory.

Therefore, using this definition, the input space complexity is:

- **Input Space (Parameter-only):** $O(1)$

2. Full-Data Definition

This broader definition considers the space taken by the entire data structure the function operates on. The function manipulates a doubly linked list pointed to by a global or class member `head`.

- If the linked list contains **n** nodes, the total memory occupied by the list is proportional to **n**.

Therefore, using this definition, the input space complexity is:

- **Input Space (Full-Data):** $O(n)$
-

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

Algorithm: Insert at Beginning (Doubly Linked List)

DINSATBEG(START, ITEM)

*(Node *)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, let's name it AVAIL.*

1. [ALLOCATE NEW NODE]

Set NEW := AVAIL.

2. [MEMORY ALLOCATED?]

If NEW = NULL, then:

Write: "Error: Memory allocation failed." and EXIT.

[End of If structure.]

3. [COPY DATA]

Set INFO[NEW] := ITEM.

4. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

a. Set PREVLINK[NEW] := NULL.

b. Set NEXTLINK[NEW] := NULL.

c. Set START := NEW.

d. Write: ITEM, "inserted at the beginning."

ELSE: [List is not empty]

a. Set PREVLINK[NEW] := NULL.

b. Set NEXTLINK[NEW] := START.

c. Set PREV[START] := NEW [Point the original first node's PREV back to the new node].

d. Set START := NEW.

e. Write: ITEM, "inserted at the beginning."

[END OF IF STRUCTURE]

5. Exit
